DVWA Stored XSS 渗透测试训练报告

一、测试概要

项目	内容
漏洞类型	存储型跨站脚本攻击 (Stored XSS)
训练平台	DVWA (Damn Vulnerable Web Application)
测试模块	XSS (Stored)
安全等级	Low / Medium / High
测试日期	2026-04-27
测试人员	Mr.X
报告编号	DVWA-XSS-S-2026-002

二、售前业务风险分析

本章节面向非技术人员，以业务视角阐述该类漏洞对企业的实际影响。

2.1 漏洞概述

存储型 XSS 与反射型 XSS 的关键区别在于：恶意脚本被**持久化存储**于服务端数据库中。当任何用户访问包含该数据的页面时，恶意脚本自动执行。攻击者无需诱导受害者点击特定链接，**所有访问该页面的用户均会受害**，危害范围远超反射型 XSS。

2.2 业务影响分析

影响维度	风险描述	严重程度
用户数据安全	所有访客的 Cookie/Session 被自动窃取，账户批量沦陷	**严重**
品牌信誉	页面内容被持久篡改，每位访客均可见恶意内容	**严重**
合规风险	大规模用户数据泄露，违反《网络安全法》《个人信息保护法》	**高**
业务连续性	持久化攻击可长期潜伏，不被发现期间持续造成损害	**严重**
经济损失	批量账户失窃、客户流失、应急处置费用、监管处罚	**高**

2.3 与反射型 XSS 的风险对比

维度	反射型 XSS	存储型 XSS
触发条件	需诱导用户点击恶意链接	用户正常访问页面即触发
影响范围	点击链接的单个用户	**所有访问该页面的用户**
持久性	一次性，链接失效即消失	**持久化存储，长期有效**
隐蔽性	URL 中可见异常参数	**无异常 URL，难以察觉**
风险等级	高	**严重**

2.4 典型攻击场景

1. **留言板/评论区投毒**: 攻击者在留言板中提交恶意脚本，管理员审核时 Cookie 被窃取
2. **蠕虫传播**: 恶意脚本自动以受害者身份发表新留言，实现类似 Samy 蠕虫的自传播效果
3. **后台管理员劫持**: 管理员查看用户提交内容时触发脚本，后台管理权限被接管
4. **水坑式批量钓鱼**: 在高流量页面植入脚本，批量收割访客凭据

三、测试环境

项目	说明
靶机平台	DVWA (Damn Vulnerable Web Application)
测试模块路径	/vulnerabilities/xss_s/
请求方式	POST
注入参数	txtName（Name 字段）/mtxMessage（Message 字段）
前端限制	Name 字段 maxlength=10，Message 字段 maxlength=50
浏览器	Chrome / Firefox
辅助工具	Burp Suite / 浏览器开发者工具 (F12)

重要提示: DVWA 的 Name 输入框存在前端 maxlength 限制，需通过以下方式绕过：

- **方法一**: 浏览器 F12 → 定位<input>元素 → 修改或删除 maxlength 属性
- **方法二**: 使用 Burp Suite 拦截请求后直接修改 txtName 参数值

四、Low 等级测试

4.1 源码分析

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
$message = trim( $_POST[ 'mtxMessage' ] );
$name = trim( $_POST[ 'txtName' ] );

// Sanitize message input
$message = stripslashes( $message );
$message = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $message);

// Sanitize name input
$name = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $name);

// Update database
$query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
$result = mysqli_query($GLOBALS["mysqli_ston"], $query )
or die( '<pre>' . mysqli_error($GLOBALS["mysqli_ston"]). '</pre>' );
```

```
}
?>
```

安全措施	分析
stripslashes()	仅去除转义反斜杠，**不防 XSS**
mysqli_real_escape_string()	仅防 SQL 注入，**不防 XSS**
htmlspecialchars()	**未使用**— 无输出编码

结论:message 和 name 两个字段**均未做任何 XSS 防护**，仅做了 SQL 注入防护。

4.2 测试步骤

Step 1: 在 DVWA 中将 Security Level 设置为 Low

Step 2: 访问 XSS (Stored) 模块，正常提交确认功能

Name: test
Message: hello world

页面显示留言内容，确认功能正常。

Step 3: 在 Message 字段注入基础 XSS Payload

Name: test
Message: <script>alert('XSS')</script>

预期结果: 提交后页面弹窗显示 "XSS"，且后续每次访问该页面均会触发弹窗。

Step 4: 在 Name 字段注入 XSS Payload

使用 F12 修改 Name 输入框的 maxlength 属性（删除或改大），然后输入：

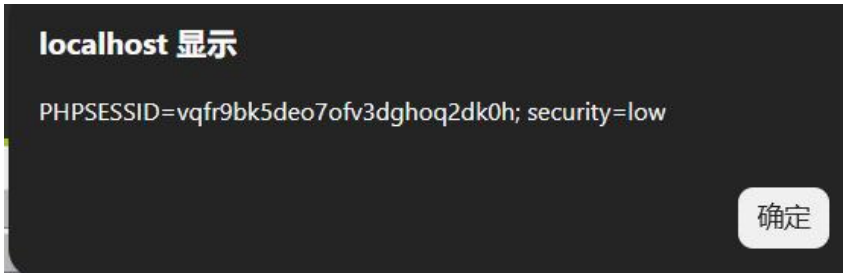
Name: <script>$alert('XSSname')$ $</script>$
Message: normal text

预期结果: Name 字段同样弹窗成功。

Step 5: 窃取 Cookie Payload

Name: test
Message: <script>alert(document.cookie)</script>

预期结果: 弹窗显示当前用户 Cookie，包含 PHPSESSID 和 security 等字段。



Step 6: 点击 "Clear Guestbook" 清除注入数据，为下一等级测试做准备。

4.3 小结

项目	结论
防御措施	仅 SQL 注入防护，无 XSS 防护
可注入字段	message 和 name 均可注入
绕过难度	无需绕过
可用 Payload	任意 XSS 向量均可直接执行

五、Medium 等级测试

5.1 源码分析

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
$message = trim( $_POST[ 'mtxMessage' ] );
$name = trim( $_POST[ 'txtName' ] );

// Sanitize message input
$message = strip_tags( addslashes( $message ) );
$message = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $message);
$message = htmlspecialchars( $message );

// Sanitize name input
$name = str_replace( '<script>', '', $name );
$name = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $name);

// Update database
$query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
$result = mysqli_query($GLOBALS["mysqli_ston"], $query )
or die( '<pre>' . mysqli_error($GLOBALS["mysqli_ston"]). '</pre>' );
}
?>
```

两个字段的防护差异:

字段	防护链	安全性
message	strip_tags()→addslashes()→mysqli_real_escape_string()→htmlspecialchars()	**安全**—标签被剥离 + 输出编码
name	str_replace('<script>', '')→mysqli_real_escape_string()	**不安全** — 仅过滤小写 <script>

结论:message 字段已被有效保护，攻击面转移到 name 字段。name 字段使用 str_replace 过滤，存在与反射型 XSS Medium 等级相同的绕过问题。

5.2 测试步骤

Step 1: 将 Security Level 设置为 Medium

Step 2: 验证 Message 字段已被防护

Name: test
Message: <script>alert('XSS')</script>

预期结果: 无弹窗。strip_tags()和 htmlspecialchars()联合阻止了攻击。

Step 3: 使用 F12 修改 Name 字段 maxlength 限制

浏览器开发者工具 → 定位<input name="txtName">→ 删除 maxlength="10"属性。

Step 4: 绕过方法一 — 大小写混用（Name 字段）

Name: <Script>alert('XSS')</Script>

Message: test

预期结果:str_replace 仅匹配全小写<script>，大小写变体直接绕过，弹窗成功。

Step 5: 绕过方法二 — 双写嵌套（Name 字段）

Name: <scr<script>ipt>alert('XSS')</scr<script>ipt>
Message: test

预期结果: 内层<script>被移除后，外层碎片拼接为完整标签，弹窗成功。

Step 6: 绕过方法三 — 替代标签（Name 字段）

Name: <img src=1 onerror=alert('XSS')>
Message: test

预期结果:<img>标签完全不在过滤范围内，弹窗成功。

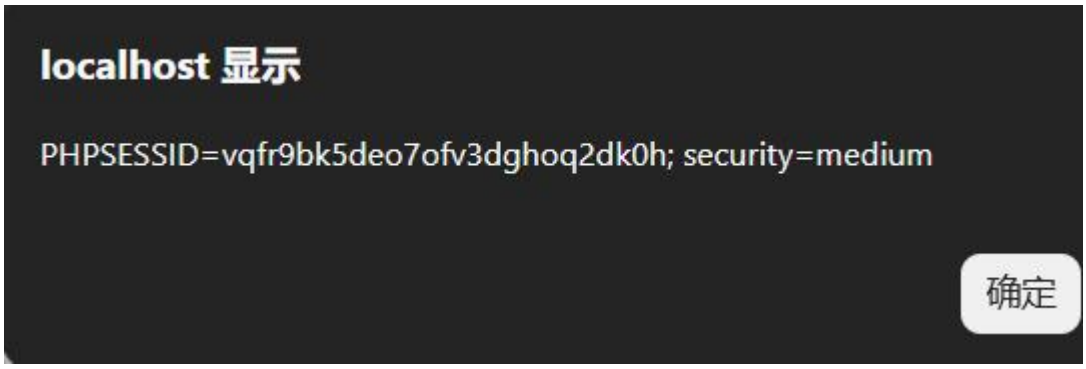
Step 7: 窃取 Cookie Payload（Name 字段）

Name: <Script>alert(document.cookie)</Script>
Message: test

或：

Name: <img src=1 onerror=alert(document.cookie)>
Message: test

预期结果: 弹窗显示当前用户 Cookie 信息。



Step 8: 清除留言板数据。

5.3 小结

项目	结论
防御措施	message 字段完整防护；name 字段仅 str_replace 过滤<script>
可注入字段	仅 name 字段
前端限制	需绕过 maxlength=10
绕过难度	低
有效绕过方法	大小写混用、双写嵌套、替代标签

六、High 等级测试

6.1 源码分析

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
$message = trim( $_POST[ 'mtxMessage' ] );
$name = trim( $_POST[ 'txtName' ] );

// Sanitize message input
$message = strip_tags( addslashes( $message ) );
$message = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $message);
$message = htmlspecialchars( $message );

// Sanitize name input
$name = preg_replace( '/<. *s. *c. *r. *i. *p. *t/i', '', $name );
$name = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $name);
```

```
// Update database
$query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
$result = mysqli_query($GLOBALS["mysqli_ston"], $query )
or die( '<pre>' . mysqli_error($GLOBALS["mysqli_ston"]). '</pre>' );
}
?>
```

两个字段的防护差异:

字段	防护链	安全性
message	strip_tags()→addslashes()→mysqli_real_escape_string()→htmlspecialchars()	安全
name	preg_replace'/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)t/i',''→mysqli_real_escape_string()	不安全 — 仅过滤<script>模式

与 Medium 的区别: 正则增加了/i 标志（大小写不敏感）和.*通配符（防嵌套），解决了大小写和双写绕过问题。但**仍然只针对<script>一种标签**，未使用 htmlspecialchars()输出编码。

6.2 测试步骤

Step 1: 将 Security Level 设置为 High

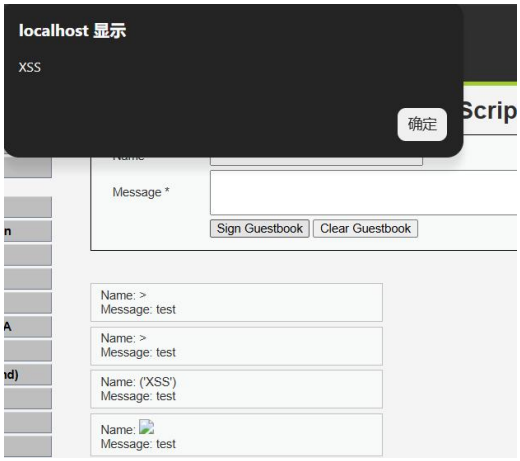
Step 2: 验证<script>相关 Payload 均已失效（Name 字段，需先绕过 maxlength）

Name: <script>alert('XSS')</script> ← 被过滤
Name: <Script>alert('XSS')</Script> ← 被过滤（/i 标志）
Name: <scr<script>ipt>alert('XSS') ← 被过滤（.* 匹配）

Step 3: 绕过方法 — IMG 标签 + 事件处理器（Name 字段）

Name: <img src=1 onerror=alert('XSS')>
Message: test

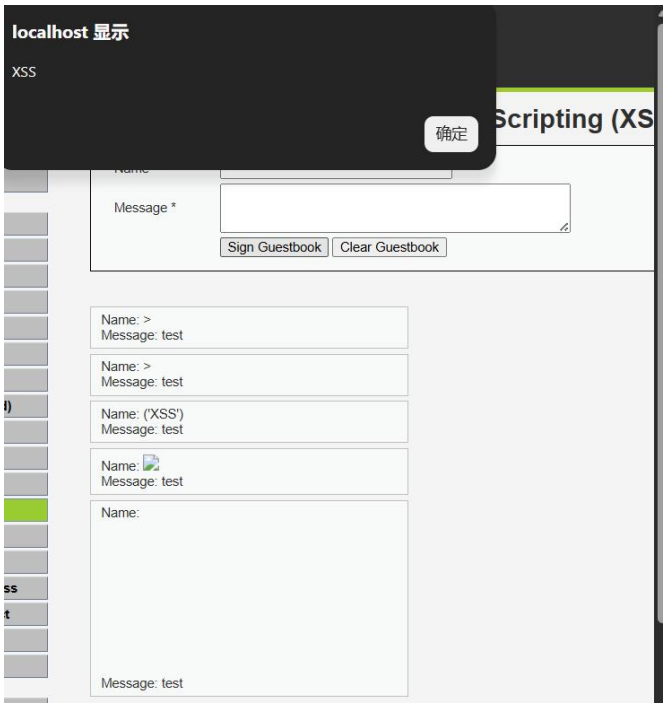
预期结果：正则仅匹配<script>模式，<img>标签不受影响，弹窗成功。



Step 4: SVG 标签绕过（Name 字段）

Name: <svg onload=alert('XSS')>
Message: test

预期结果：弹窗成功。



Step 5：其他可用 Payload（Name 字段）

<body onload=alert('XSS')>
<iframe src="javascript:alert('XSS')">

<input onfocus=alert('XSS') autofocus>

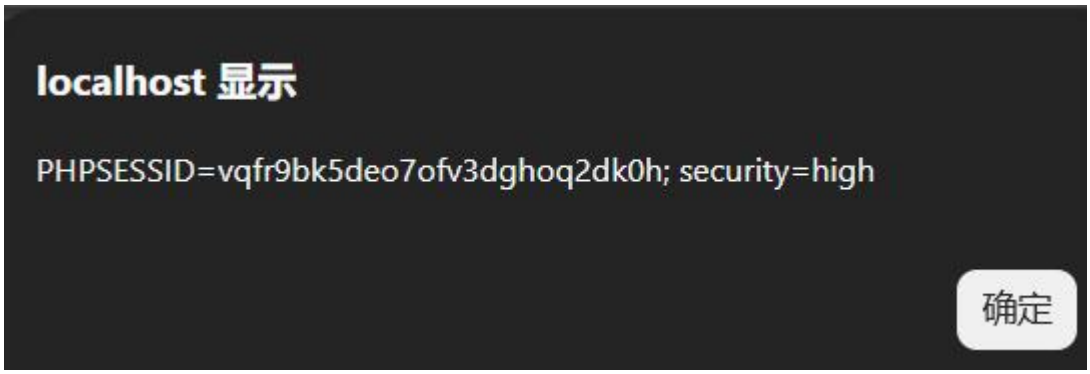
Step 6: 窃取 Cookie Payload（Name 字段）

Name: <img src=1 onerror=alert(document.cookie)>
Message: test

或：

Name: <svg onload=alert(document.cookie)>
Message: test

预期结果: 弹窗显示当前用户 Cookie 信息。



Step 7: 清除留言板数据。

6.3 小结

项目	结论
防御措施	message 字段完整防护；name 字段正则过滤所有<script>变体
可注入字段	仅 name 字段
前端限制	需绕过 maxlength=10
绕过难度	低-中
有效绕过方法	使用不含 script 关键字的标签（img/svg/body/iframe 等）

七、三等级横向对比

7.1 防护措施对比

字段	Low	Medium	High
message	stripslashes+ SQL 转义	strip_tags+addslashes+ SQL 转义 **+htmlspecialchars**	同 Medium
name	仅 SQL 转义	str_replace('<script>','')+ SQL 转义	preg_replace 正则 + SQL 转义

7.2 攻击面对比

维度	Low	Medium	High
可注入字段	message+name	仅 name	仅 name
前端绕过	可选	**必须**（maxlength）	**必须**（maxlength）
核心绕过思路	直接注入	大小写/嵌套/替代标签	替代标签+事件处理器
绕过难度	无	低	低-中
根本问题	完全无 XSS 防护	name 字段黑名单不完整	name 字段仅防一种标签

核心结论: 三个等级中 message 字段的防护逐步加强，Medium/High 已通过 htmlspecialchars() 实现安全输出。但 name 字段在所有等级中均未做输出编码，仅依赖黑名单过滤，始终存在可被绕过的 XSS 漏洞。

八、修复建议与代码示例

8.1 正确的修复方案（对应 DVWA Impossible 等级思路）

```
<?php

if( isset( $_POST[ 'btnSign' ] ) ) {
// Check Anti-CSRF token
checkToken( REQUEST['usertoken'],_SESSION[ 'session_token' ], 'index.php' );
```

```
$message = trim( $_POST[ 'mtxMessage' ] );
$name = trim( $_POST[ 'txtName' ] );

// Sanitize message input
$message = stripslashes( $message );
$message = mysqli_real_escape_string($GLOBALS["mysqli_ston"], $message);
$message = htmlspecialchars( $message, ENT_QUOTES, 'UTF-8' );

// Sanitize name input — 核心修复点
$name = stripslashes( $name );
$name = mysqli_real_escape_string($GLOBALS["mysqliston"], $name);
$name = htmlspecialchars( $name, ENT_QUOTES, 'UTF-8' );

// Prepared statement to prevent SQL injection
stmt =GLOBALS["mysqliston"] -> prepare(
"INSERT INTO guestbook ( comment, name ) VALUES ( ?, ? );"
);
$stmt->bind_param('ss', $message, $name);
$stmt->execute();
$stmt->close();
}

// Generate Anti-CSRF token
generateSessionToken();
?>
```

8.2 修复要点

修复项	说明
htmlspecialchars()应用于两个字段	message 和 name 均需输出编码，不可遗漏
参数化查询	用 Prepared Statement 替代字符串拼接，彻底防止 SQL 注入
Anti-CSRF Token	防止攻击者通过 CSRF 代替受害者提交恶意留言
ENT_QUOTES 参数	同时编码单引号和双引号
UTF-8 编码指定	防止多字节字符编码绕过

8.3 防御体系建议

层级	措施	说明

层级	措施	说明
代码层	htmlspecialchars()输出编码	根本解决方案，所有输出字段必须覆盖
数据库层	Prepared Statement 参数化查询	同时防御 SQL 注入
HTTP 层	Content-Security-Policy: script-src 'self'	禁止内联脚本执行
会话层	Anti-CSRF Token	防止跨站提交恶意数据
架构层	模板引擎自动转义（如 Twig/Blade）	从框架层面杜绝遗漏
测试层	自动化 XSS 扫描集成到 CI/CD	持续检测回归漏洞

九、训练总结

本次基于 DVWA Stored XSS 模块的训练覆盖了三个安全等级的攻防对抗：

1. **Low**: 两个字段均无 XSS 防护，理解存储型 XSS 的持久化危害本质
2. **Medium**:message 字段已修复，攻击面转移到 name 字段；理解黑名单 str_replace 的局限性 + 前端 maxlength 绕过
3. **High**: 正则过滤阻止了所有<script>变体，但无法防御事件处理器类攻击向量

关键训练收获:

- 存储型 XSS 的影响范围远大于反射型，因其**持久化存储、自动触发、批量影响**
- 安全防护必须**覆盖所有输入字段**，Medium/High 等级中 message 已修复但 name 被遗漏，体现了"木桶效应"
- 前端限制（maxlength）**不可作为安全措施**，攻击者可轻松绕过
- **输出编码**（htmlspecialchars）是防御 XSS 的根本手段，黑名单过滤永远不够完善